



Improved QPSO with Darwinian evolution theory for three-dimensional path planning of robots

Mengfan Wang^{a,b}, Xinglin Chen^{a,b}, Siyu Liu^{a,b}, Sisindisiwe Nomalanga Ncube^{a,b},
Ruotong Ming^c, Fuqiang Liu^{a,b}, Jun Luo^{a,b}, Huayan Pu^{a,b}

^a State Key Laboratory of Mechanical Transmission for Advanced Equipment, Chongqing University, Chongqing, 400044, China

^b College of Mechanical and Vehicle Engineering, Chongqing University, Chongqing, 400044, China

^c College of Engineering and Applied Science, University of Cincinnati, Cincinnati, OH 45221, USA

ARTICLE INFO

Keywords:

Quantum-behaved particle swarm optimization
Competitive-assimilation strategy
Coyote Optimization Algorithm
Adaptive law
Three-dimensional path planning

ABSTRACT

Effective three-dimensional (3D) path planning is crucial for robots to achieve missions in complex environments, where swarm optimization algorithms, which are important methods in path planning, face challenges such as local optimum and high sensitivity to initial particle positions. Therefore, this paper proposes a competitive-assimilation strategy based on Darwinian evolution theory and combines it with the Quantum-behaved Particle Swarm Optimization (QPSO) algorithm to solve the challenges of optimization in 3D path planning. Simultaneously, a novel adaptive law and the Coyote Optimization Algorithm (COA) are both introduced into the algorithm for further strengthening the local search capability. By comparing with the latest related algorithms in simulation experiments and statistical validation tests, the two proposed methods demonstrate superior path quality (achieving mean value reductions of 1.51% and 3.44%, respectively) and enhanced algorithmic stability (with standard deviation reductions of 55.41% and 86.88%, respectively). These results substantiate the effectiveness and superiority of the proposed methodology in 3D path planning applications.

1. Introduction

As people increasingly focus on exploring ocean environments, Autonomous Underwater Vehicles (AUVs) [1], one of the important components of robots, have received significant attention and research within the realm of intelligent unmanned systems. They play increasingly important roles in underwater tasks such as rescue, surveying, data collection, and fisheries. However, the complexity and uncertainty of the ocean environment present significant challenges for the motion planning of AUVs. In response to these challenges, three-dimensional (3D) path planning has become a key technology for AUVs to execute tasks in complex ocean conditions [2].

Current research in path planning primarily focuses on traditional search algorithms, intelligent bio-inspired algorithms, and machine learning techniques [3,4]. Zhang et al. [5] combined the A* algorithm with the artificial potential field (APF) algorithm, addressing dynamic constraints and enhancing the speed of the A* algorithm. In the field of path planning based on machine learning, Chu et al. [6] introduced a deep feedback learning (DRL) approach using a double deep Q-Network (DDQN), which incorporates an advanced convolutional neural network. Similarly, Yang et al. [7] presented an n -step priority

DDQN path planning method. Moreover, there are emerging studies such as the DQN-QPSO algorithm based on DQN and Quantum-behaved Particle Swarm Optimization (QPSO) [8], as well as a reinforcement learning (RL) based Tuna Swarm Optimization (TSO) algorithm [9], which combined machine learning with optimization algorithms. However, traditional search algorithms mentioned above suffer from slow search speeds in multidimensional problems, and there are difficulties in training machine learning in the complex path planning field [2].

As principal methodologies for tackling real-world engineering optimization challenges [10,11], intelligent optimization algorithms have been widely applied in domains such as path planning. Scholars have proposed different methods such as the two-dimensional (2D) path planning based on Ant Colony Optimization (ACO) [12–14], Adaptive Genetic Algorithm (GA) [15], improved Sparrow Search Algorithm (SSA) [16], enhanced Whale Optimization Algorithm (WOA) [17], hybrid Particle Swarm Optimization (PSO) [18], Nature-inspired Greylag Goose Optimization (GGO) [19], and hybrid Grey Wolf Optimization (GWO) algorithms [20,21]. In the research of typical algorithm fusion, Wen et al. [22] enhanced the integration of GA, ACO, and Simulated Annealing (SA) to address slow convergence issues of typical heuristic

* Corresponding author.

E-mail address: liufq@cqu.edu.cn (F. Liu).

<https://doi.org/10.1016/j.swevo.2026.102334>

Received 14 March 2025; Received in revised form 2 June 2025; Accepted 10 February 2026

Available online 16 February 2026

2210-6502/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

algorithms. Myriam et al. [23] proposed a hybrid of PSO and Al-Biruni Earth Radius (BER) optimization algorithm. Further, Li et al. [24] performed a comparative study on the effectiveness of GA, PSO, ACO, Artificial Bee Colony (ABC), and Wolf Pack Algorithm (WPA) for 3D path planning of AUVs, finding PSO more effective.

Based on PSO, the improved compression factor PSO (ICFPSO) [25] was explored to enhance PSO. Lu et al. [26] addressed the tendency of PSO towards local optima by improving inertia and cognitive weights, and incorporating the average optimal position into the velocity update formula. Concurrently, Huang et al. [27] developed an adaptive PSO that used differential evolution operators to adjust parameters, cylindrical vectors, and challenge operators. Gupta and Verma [28] integrated the Coyote Optimization Algorithm (COA) with PSO, creating HCPSOA, which capitalized on the global exploration strengths of PSO and the local exploration strengths of COA. Yu et al. [29] introduced the SDPSO algorithm for unmanned aerial vehicles, combining PSO with SA and incorporating a dimensional learning strategy. Indeed, even enhanced PSO algorithms like QPSO [30] still have a non-negligible probability of getting trapped in the local optimum. Therefore, Li et al. [31] introduced binary strategy; Lu et al. [32] proposed the method of particle initialization and updating strategy. In summary, the aforementioned research has addressed the issues of local optimum and slow convergence of the algorithm to some extent. However, when dealing with high-dimensional complex problems, the problem of falling into a local optimum persists. This limitation primarily stems from swarm optimization algorithms' sensitivity to initialization and inherent iterative update mechanisms, which significantly degrade performance and solution consistency in PSO-like methodologies. Therefore, the research for resolving initialization sensitivity and local optimum not only can enhance its theoretical foundation but also provides transferable insights for improving analogous swarm-based and stochastic optimization approaches, demonstrating both academic significance and practical value in optimization problems.

Currently, intelligent bio-inspired algorithms, particularly the PSO algorithm, are the primary methods for 3D path planning in AUVs, offering substantial benefits. However, as a kind of swarm optimization algorithms, PSO is prone to a local optimum and is significantly affected by its initial settings. To address these challenges and enhance its efficacy in 3D path planning for AUVs, this paper makes several key contributions:

(1) A competitive-assimilation strategy based on Darwinian evolution theory is proposed to solve the shortcoming of swarm optimization algorithms.

(2) A novel adaptive law is introduced for adjusting the Contraction-Expansion (CE) coefficient of QPSO, resulting to the development of the Competitive-Assimilation Adaptive QPSO (CAAQPSO) algorithm.

(3) Enhanced CAAQPSO (CAA*QPSO) is generated by integrating the CAAQPSO and COA, enhancing the algorithm's local search capabilities further.

The paper is organized as follows: Section 2 establishes the 3D path planning model. Section 3 elaborates on the main contributions of this paper, including the proposal of the competitive-assimilation strategy, development of the CAAQPSO and CAA*QPSO, establishment of the evaluation function, and along with the path planning process. Experiments the presentation are detailed in Section 4. Finally, Section 5 provides conclusions.

2. Preliminaries and problem formulation

To comprehensively describe 3D path planning research, this section introduces both the 3D ocean model and path representation methods required for AUV path planning.

Establishing corresponding coordinate systems can facilitate a better understanding of the planning process. Based on the operational environment of AUVs, this paper establishes the body-fixed coordinate system $o_b x_b y_b z_b$ (Front-Right-Down) and the world coordinate system $O_n X_n Y_n Z_n$ (North-East-Down) using the laboratory robot ROV to describe the robot's motion state, as shown in Fig. 1.

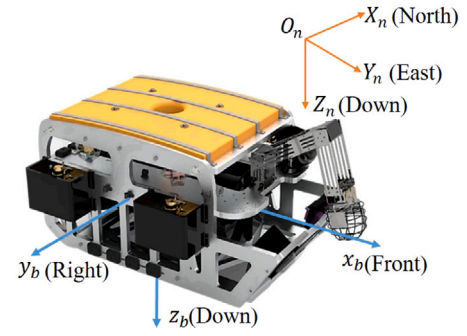


Fig. 1. The AUV model.

2.1. 3D ocean environmental model

To conduct path planning research in complex ocean environments, establishing realistic environmental representations based on actual missions is imperative. Therefore, this subsection establishes a 3D ocean environmental model incorporating seafloor topography, suspended obstacles, and ocean currents.

2.1.1. Ocean obstacle environment model

In AUV operational environments, both seafloor topography and suspended obstacles must be considered. The exponential function method [33] is employed to randomly generate a 3D underwater environment model to meet the requirement for establishing natural underwater mountain models with higher elevations. This is presented as a mathematical model

$$Z(x, y) = \sum_{i=1}^n h_i \exp \left[- \left(\frac{x - x_i}{x_{si}} \right)^2 - \left(\frac{y - y_i}{y_{si}} \right)^2 \right] \quad (1)$$

where n represents the total number of peaks; (x_i, y_i) denotes the central coordinates of the i th peak; h_i , x_{si} , and y_{si} represent the terrain control parameters, which respectively control the height, x -direction slope (decay amount), and y -direction slope (decay amount) of the i th peak.

Furthermore, static suspended obstacles in space are expanded into spheres. The expression in Cartesian coordinates is as follows

$$\begin{cases} X_c(i) = R(i)\cos(\theta)\cos(\phi) + X_0(i) \\ Y_c(i) = R(i)\cos(\theta)\sin(\phi) + Y_0(i) \\ Z_c(i) = R(i)\sin(\theta) + Z_0(i) \end{cases} \quad (2)$$

where $R(i)$ represents the radius of the i th sphere; $(X_0(i), Y_0(i), Z_0(i))$ denotes the center coordinate of the i th sphere; $(X_c(i), Y_c(i), Z_c(i))$ is the current surface coordinates of the i th sphere; θ and ϕ denote the polar and azimuthal angles, respectively.

2.1.2. Ocean current model

In real ocean environments, the influence of complex ocean currents on AUV motion is considerable. Therefore, it is essential in path planning and motion control studies to establish an appropriate ocean current environment. Relative to the sea surface, the oceanic environment can be considered quasi-steady over a certain period. Taking the initial flow field vector as $(V_{x0}, V_{y0}, 0)$, establishing a Lamb vortex environment model [34] is

$$\begin{cases} V_{xi}(\vec{r}) = -\Gamma \frac{y-y_0}{2\pi(\vec{r}-\vec{r}_0)} \left(1 - e^{-\left(\frac{(\vec{r}-\vec{r}_0)^2}{\zeta^2}\right)} \right) \\ V_{yi}(\vec{r}) = \Gamma \frac{x-x_0}{2\pi(\vec{r}-\vec{r}_0)} \left(1 - e^{-\left(\frac{(\vec{r}-\vec{r}_0)^2}{\zeta^2}\right)} \right) \\ \omega_i(\vec{r}) = \frac{\Gamma}{\pi\zeta} e^{-\left(\frac{(\vec{r}-\vec{r}_0)^2}{\zeta^2}\right)} \end{cases} \quad (3)$$

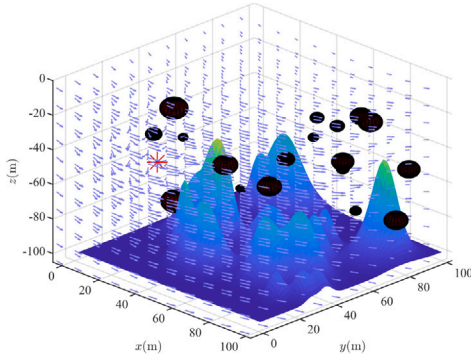


Fig. 2. The ocean environment model.

where $V_{xi}(\vec{r})$, $V_{yi}(\vec{r})$, and $\omega_i(\vec{r})$ represent the horizontal velocity component, vertical velocity component, and vorticity of the i th vortex, respectively; $\vec{r}_0$, ζ , and Γ denote the center position, radius, and strength of the vortex, respectively.

The velocity components of the initial flow field and the superimposed velocity components of multiple vortices yield (V_x, V_y, ω)

$$\begin{cases} V_x = V_{x0} + V_{x1} + V_{x2} + \dots + V_{xn} \\ V_y = V_{y0} + V_{y1} + V_{y2} + \dots + V_{yn} \\ \omega = \omega_1 + \omega_2 + \dots + \omega_n \end{cases} \quad (4)$$

As shown in Fig. 2, the final environment model is generated, in which the sphere represents the suspended obstacle, the arrow means the flow field direction, and the asterisk is the vortex center.

2.2. 3D path representation

For path description, this paper considers the intermediate path points (x_k, y_k, z_k) between the starting point (x_0, y_0, z_0) and the endpoint (x_N, y_N, z_N) as the parameters to be determined. To avoid generating poor paths due to redundant clustering of path points, the $X-Z$ coordinate planning plane is uniformly divided along the Y -axis. The generated path is then interpolated into n path points using cubic spline interpolation, followed by smoothing processing to make the final path more consistent with underwater navigation rules. The path is a parametric equation as follows

$$F_i(\delta) = (x_i(\delta), y_i(\delta), z_i(\delta)) \quad (5)$$

$$\begin{cases} x_i(\delta) = a_{x,i}(\delta - \delta_i)^3 + b_{x,i}(\delta - \delta_i)^2 + c_{x,i}(\delta - \delta_i) + d_{x,i} \\ y_i(\delta) = a_{y,i}(\delta - \delta_i)^3 + b_{y,i}(\delta - \delta_i)^2 + c_{y,i}(\delta - \delta_i) + d_{y,i} \\ z_i(\delta) = a_{z,i}(\delta - \delta_i)^3 + b_{z,i}(\delta - \delta_i)^2 + c_{z,i}(\delta - \delta_i) + d_{z,i} \end{cases} \quad (6)$$

where $a_{\cdot,i}$, $b_{\cdot,i}$, $c_{\cdot,i}$, and $d_{\cdot,i}$ represent the unknown coefficients to be determined; $\delta \in [\delta_i, \delta_{i+1}]$ ($i = 0, 1, \dots, N-1$) denotes the path segments, where δ_i is uniformly distributed in the interval $[0, 1]$.

2.3. Problem formulation

In addressing the 3D path planning problem of AUVs in complex underwater environments, the comparison [24] demonstrates that PSO is relatively more suitable for underwater path planning of AUVs. In traditional PSO [35], the particle population is defined as M particles with a dimensionality of D , and the boundary conditions are [LB, UB]. The particle velocity v_i , position x_i , individual best position P_i , and global best position G are updated according to the following rules

$$v_{ij}(t+1) = \omega v_{ij}(t) + c_1 q_1 (P_{ij}(t) - x_{ij}(t)) + c_2 q_2 (G_j(t) - x_{ij}(t)) \quad (7)$$

$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1) \quad (8)$$

$$v_{ij}(t+1) = \begin{cases} v_{\min}, v_{ij} \leq v_{\min} \\ v_{\max}, v_{ij} \geq v_{\max} \end{cases} \quad (9)$$

$$x_{ij}(t+1) = \begin{cases} x_{\min}, x_{ij} \leq x_{\min} \\ x_{\max}, x_{ij} \geq x_{\max} \end{cases} \quad (10)$$

$$P_i(t+1) = \begin{cases} x_i(t+1), & J(x_i(t+1)) \leq J(P_i(t)) \\ P_i(t), & J(x_i(t+1)) > J(P_i(t)) \end{cases} \quad (11)$$

$$G(t) = \arg \min J(P_i(t)) \quad (12)$$

where ij denotes the j th dimension of the i th particle; $t = 0, 1, \dots, T$ represents the current iteration number, where $t = 0$ indicates population initialization, and T is the total number of iterations; ω , c_1 , and c_2 represent the inertia weight, individual cognitive weight, and social cognitive weight, respectively; q_1 and q_2 denote random numbers in the range $[0, 1]$; $J(\cdot)$ denotes the evaluation function.

The update rules of QPSO [30], which incorporate quantum behavior, are not constrained by velocity and inertia. Instead, the particle position is updated according to the following rules

$$P_m(t) = \frac{1}{M} \sum_{i=1}^M P_i(t) \quad (13)$$

$$\psi(t) = \psi_{\min} + (\psi_{\max} - \psi_{\min}) \frac{T-t}{T} \quad (14)$$

$$p_{ij}(t+1) = \varphi P_{ij}(t) + (1 - \varphi) G_j(t) \quad (15)$$

$$x_{ij}(t+1) = p_{ij}(t+1) \pm \psi \left| P_{mj}(t) - x_{ij}(t) \right| \ln \frac{1}{u} \quad (16)$$

where p_i represents the center of particle motion; φ and u are random numbers in the range $(0, 1]$; ψ is the CE coefficient; P_m denotes the average optimal position. The pseudo-code of QPSO is shown in Algorithm 1.

Algorithm 1 The QPSO

Input: $M, D, T, \psi_{\max}, \psi_{\min}, LB, UB$

Output: $G, J(G)$

```

1: Initialize population;
2: for  $t = 1$  to  $T$  do
3:   Update  $P_m$  from (13);
4:   Update  $\psi$  from (14);
5:   for  $i = 1$  to  $M$  do
6:     for  $j = 1$  to  $D$  do
7:       Update  $p_{ij}$  from (15);
8:       Update  $x_{ij}$  from (16);
9:       Judge boundary of position from (10);
10:    end for
11:    Update  $P_i$  and  $G$  from (11) and (12);
12:  end for
13: end for

```

Currently, in addressing path planning problems, QPSO still faces challenges such as susceptibility to a local optimum that leads to suboptimal solutions, sensitivity to particle initialization, and poor precision [36]. To solve the aforementioned issues, this paper proposes an improved QPSO algorithm that includes the development of a competitive-assimilation strategy based on Darwinian evolution theory and adaptive adjustment of the CE coefficient for different updated states. Based on this research, the advantages of the COA are combined with those of the CAAQPSO algorithm to create the CAA*QPSO algorithm, which possesses enhanced capabilities for both global and local search.

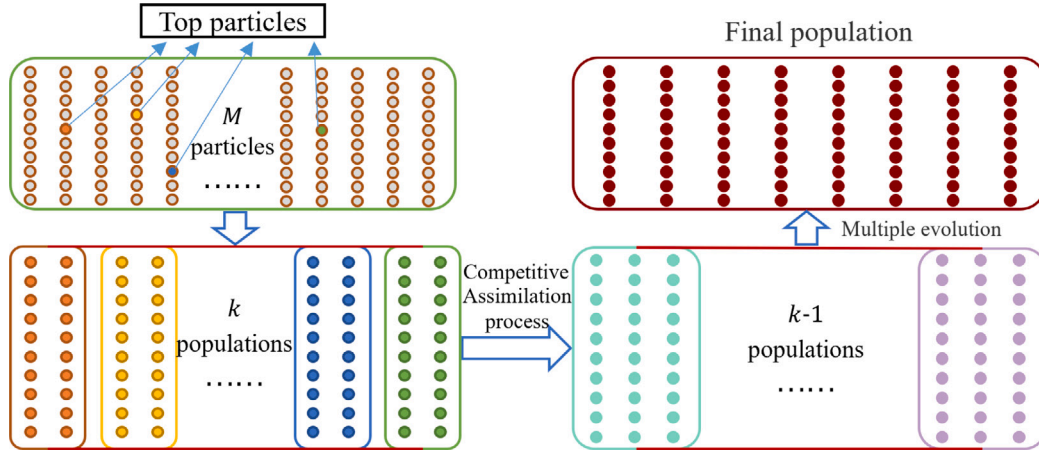


Fig. 3. The schematic diagram of the competitive-assimilation strategy.

3. The main work

Firstly, it proposes the competitive-assimilation strategy, and then develops the CAAQPSO algorithm. Subsequently, by making full use of the local search ability of the COA, the CAA*QPSO is proposed. Finally, it designs an evaluation function and constraints for 3D path planning of AUVs in complex environments.

3.1. Competitive-assimilation strategy

Drawing upon the principles of Darwinian evolution theory, this paper proposes a novel population allocation principle and renewal rule, that exhibit remarkable efficacy in addressing population diversity and escaping local optimization.

Specifically, following the initialization of the population, the survival competitiveness of each particle is evaluated through evaluation function. The top k particles, identified based on their evaluation values, are designated as leader candidates, subsequently assuming leadership roles within their respective populations. The worst L particles are then eliminated to ensure that the remaining number M_0 of particles is evenly distributed, giving rise to the initial k populations. Eventually, k initial populations containing $m(1)$ particles are formed.

$$L(1) = M - k \left\lfloor \frac{M}{k} \right\rfloor \quad (17)$$

$$M_0(1) = M - k - L \quad (18)$$

$$m(1) = \frac{M_0}{k(1)} + 1, (m \geq \xi) \quad (19)$$

where 1 represents the first time population distribution; $\xi = 20$ is constrained to the minimum particle number to ensure adequate population size for global optimization capability.

Secondly, an evolution rule is devised for the acquired populations. Initially, individual optimization and global optimization of multiple populations are initialized without altering particle positions, ensuring the superiority of leading particles. Subsequently, competition of populations is implemented based on evolutionary theory, and multiple populations are concurrently updated for τ iterations. Based on evaluation function values, the k initial populations are competitively ranked, and the weakest population is disbanded. Subsequently, L inferior particles are then eliminated, and the positions of the remaining particles are reset to ensure randomness and uniformity when the dominant population absorbs the inferior one. The m particles of the resulting new k populations comprise both original high-quality particles and newly integrated initialization particles. This facilitates

competitive assimilation within and among populations, completing the initial evolution and development among them.

$$\tau = \left\lceil \frac{T}{k(1)} \right\rceil \quad (20)$$

$$L(q+1) = m(q) - k(q) \left\lfloor \frac{m(q)}{k(q) - 1} \right\rfloor \quad (21)$$

$$m(q+1) = m(q) + \left\lfloor \frac{m(q)}{k(q) - 1} \right\rfloor \quad (22)$$

$$k(q+1) = k(q) - 1 \quad (23)$$

where $q = 1, 2, \dots, k$ is the current number of evolutions.

After undergoing q rounds of competitive-assimilation selection, the population underwent further evolution and merging, resulting in the acquisition of the optimal population. Subsequently, the optimal population underwent τ iterations according to steps 6 to 14 from Algorithm 2 to determine the current optimal solution as shown in Fig. 3.

Remark 1. The competitive-assimilation strategy based on Darwinian evolution theory is suitable for swarm optimization algorithms to avoid falling into local optimal problem, which is verified on QPSO algorithm in this paper.

3.2. Competitive-assimilation adaptive QPSO

This section introduces the formation of the CAAQPSO, including the development of a novel adaptive law and the utilization of competitive-assimilation strategy.

In QPSO algorithm, the CE coefficient ψ serves as a crucial parameter directly influencing the search speed and convergence rate of particles [36]. Choosing an unsuitable value for ψ can lead to several issues, including slow convergence, convergence to the local optimum, deviation from the global optimum, and insufficient exploration of the search space. To mitigate these challenges, a novel adaptive law is developed and integrated for dynamically adjusting the CE coefficient ψ , thereby enhancing the optimization capability.

The present state of particle iterations can be characterized by iteration duration and particle aggregation degree. Therefore, the iteration coefficient w and the aggregation degree coefficient s are devised

$$w(t) = \frac{t}{\tau} \quad (24)$$

$$s(t) = \frac{J(G(t))}{\frac{1}{m} \sum_{i=1}^m J(x_i(t))} \quad (25)$$

where $w(t), s(t) \in [0, 1]$; and a smaller value of $s(t)$ corresponds to greater particle diversity.

By designing ψ_0 , the range of CE coefficient is approximately limited to a range. The adaptive CE coefficient ψ is designed as follows

$$\psi_0 = \exp(\rho_1 w(t)) - \sin(\rho_2 \pi w(t)) \quad (26)$$

$$\psi = \begin{cases} \max\{\psi_0, \psi_0 + \Xi_1 \exp(\zeta_1 s(t) - \iota_1 w(t)) \cos(\kappa_1 \pi w(t))\}, & w(t) \in [0, \Psi_1] \\ \min\{\psi_0, \psi_0 + \Xi_2 \exp(\zeta_2 s(t) - \iota_2 w(t)) \cos(\kappa_2 \pi w(t))\}, & w(t) \in (\Psi_1, \Psi_2) \\ \psi_0 + \Xi_3 \exp(\zeta_3 s(t) - \iota_3 w(t)) \cos(\kappa_3 \pi w(t)), & w(t) \in [\Psi_2, 1] \end{cases} \quad (27)$$

where $\rho_{(\cdot)}$, $\Xi_{(\cdot)}$, $\zeta_{(\cdot)}$, $\iota_{(\cdot)}$, and $\kappa_{(\cdot)}$ are the parameters of the adaptive law, respectively. The parameters can be determined according to the findings in [36]. $\Psi_1, \Psi_2 \in [0, 1]$ is based on experience. $w(t) \in [0, \Psi_1]$ is considered the early iteration phase. A higher aggregation degree $s(t)$ corresponds to a stronger global search capability, facilitating escape from the local optimum. $w(t) \in (\Psi_1, \Psi_2)$ represents the mid-iteration phase, where a higher aggregation degree $s(t)$ leads to a smaller ψ , striking a balance between global and local search capabilities while ensuring fast convergence. When $w(t) \in [\Psi_2, 1]$, it denotes the late iteration phase. During this stage, minor adjustments in ψ_0 promote enhanced local search capabilities.

The proposed competitive-assimilation strategy in CAAQPSO achieves optimization through three core modifications: altering the population initialization mechanism, redefining the utilization approach for updated information, and redesigning the population renewal process. The novel adaptive CE coefficient formula further enhances this framework by replacing the original mathematical structure while maintaining computational efficiency. Crucially, these architectural innovations induce only marginal increases in computational overhead. Comprehensive time complexity analysis within the experimental validation section will quantitatively substantiate these efficiency claims. The final pseudo-code of CAAQPSO is shown as Algorithm 2.

3.3. The CAA*QPSO based on COA

The COA [37] is a population optimization algorithm. It has strong local search ability but runs slowly when the search space is large. Inspired by Gupta et al. [28], the excellent local search ability of the COA can further improve the optimization ability of the CAAQPSO algorithm. Because of the high aggregation of particles in the later stage, updating the COA for multiple identical particles will greatly reduce the efficiency of the algorithm. Therefore, this paper proposes a memory mechanism to overcome the slow convergence of the COA. It identifies the same particles to avoid repeated processing, thus forming CAA*QPSO algorithm on the basis of CAAQPSO.

The population is divided into N_p packs containing N_c individuals. The social condition sc of i coyote in the j th pack of k dimension is initialized as

$$sc_{i,k}^j(t) = LB_k + \lambda(UB_k - LB_k) \quad (28)$$

where $\lambda \in [0, 1]$. The probability of individual N_c joining other groups is P_l

$$P_l = 0.005N_c^2. \quad (29)$$

The social condition sc of each coyote is evaluated as

$$P_i^j(t) = J(sc_{i,k}^j(t)). \quad (30)$$

Algorithm 2 The CAAQPSO

Input: Map parameters, CAAQPSO parameters
Output: Global optimal solution

- 1: Initialize population;
- 2: Evenly distribute the remaining particles M_0 to k leader candidates, and obtain k initial populations;
- 3: **for** $q = [k, k-1, \dots, 1]$ **do**
- 4: **for** $n = 1$ to q **do**
- 5: Initialize multi-population (without changing particle position);
- 6: **for** $t = 1$ to τ **do**
- 7: Update P_m from (13);
- 8: Update $w(t)$ and $s(t)$ from (24) and (25);
- 9: Update ψ from (27);
- 10: **for** $i = 1$ to m **do**
- 11: Update p_{ij} and x_{ij} from (15) and (16);
- 12: Update P_i and G from (11) and (12);
- 13: **end for**
- 14: **end for**
- 15: **end for**
- 16: **if** $q > 1$ **then**
- 17: Compare populations;
- 18: Disintegrate the worst population and eliminate particles l ;
- 19: Initialize captive particles and evenly distribute to $q-1$ populations;
- 20: **end if**
- 21: **end for**

The optimal solution in each pack is

$$G^j(t) = \arg \min J(P_i^j(t)), \quad (31)$$

where G^j and other individuals jointly influence the social behavior, which is called the cultural tendency of the group

$$ct_i^j(t) = \text{median}(sc_{i,k}^j(t)), \quad \forall i \in \{1, 2, \dots, N_c\}. \quad (32)$$

Finally, the new individual social condition is updated by

$$new_sc_i^j(t) = sc_i^j(t) + r_1(ct_i^j(t) - sc_{in_1}^j(t)) + r_2(P_i^j(t) - sc_{in_2}^j(t)) \quad (33)$$

where r_1 and r_2 are random numbers from $[0, 1]$; in_1 and in_2 are two random coyotes in the i th pack. Update the social conditions according to

$$sc_i^j(t+1) = \begin{cases} new_sc_i^j(t), & \text{if } J(new_sc_i^j(t)) < P_i^j(t) \\ sc_i^j(t), & \text{otherwise} \end{cases}. \quad (34)$$

The COA considers the birth and death of an individual, expressed as age. The new individual is influenced by the social conditions and both parents:

$$pup_k^j = \begin{cases} sc_{n_1,k}^j, & r < \frac{1}{D} \quad \text{or} \quad k = k_1 \\ sc_{n_2,k}^j, & r \geq \frac{1}{D} + \frac{1-\frac{1}{D}}{2} \quad \text{or} \quad k = k_2 \\ R, & \text{otherwise} \end{cases} \quad (35)$$

where n_1 and n_2 represent two random parents in the j th pack; D and R are dimensions and random candidate solutions, respectively; k_1 and k_2 are two random dimensions; and r is as random number inside $[0, 1]$. According to the comparison between $J(pup_k^j)$ and the worst solution, the new individual joins the population instead of the worst solution when the $J(pup_k^j)$ is less than the worst solution.

The CAA*QPSO is the way to improve CAAQPSO as shown in Algorithm 3, where q , P_i , and G are the same as CAAQPSO, and are current population quantity, individual best position, and global best position,

respectively. A selection set is added to reduce the influence of repeated search of COA. Based on subpopulation quantity and particle iteration states, the optimal timing for local search enhancement is determined to maximize utilization of the CAAQPSO update mechanism. When the population is completely competitive and assimilated into a population, and the individual cannot find a better solution, COA is used to further search for the local optimum.

Algorithm 3 The updating rule of CAA*QPSO

Input: N_p, N_c, T, D, LB, UB

Output: G, J

```

1: Processed=[ ];
2: if  $q > 1$  or  $P_i > G$  then
3:   Update  $p_{ij}$  and  $x_{ij}$  from (15) and (16);
4:   Update  $P_i$  and  $G$  from (11) and (12);
5: else
6:   Current =  $P_i$ ;
7:   if Current not in Processed then
8:     Initialize  $N_p$  with  $N_c$  coyotes from (28);
9:     Evaluate the fitness of each coyote from (30);
10:    for  $t = 1$  to  $T$  do
11:      for  $j = 1$  to  $N_p$  do
12:        Evaluate the optimal solution  $G^j(t)$  from (31);
13:        Evaluate the cultural tendency from (32);
14:        for  $i = 1$  to  $N_c$  do
15:          Update social condition from (33);
16:          Update individual optimality from (34);
17:        end for
18:        Generate new individual from (35);
19:      end for
20:      Transfer between packs by (29);
21:      Update the coyotes ages;
22:    end for
23:    Processed= [Processed, Current];
24:  end if
25: end if
  
```

3.4. Evaluation function

To enable safe, efficient, and energy-efficient autonomous underwater operations of AUVs in complex underwater environments, this paper designs an evaluation function $J(F)$, which complies with AUV navigation rule [25], and it is expressed as

$$J(F) = \mu_1 J_O + \mu_2 J_L + \mu_3 J_K + \mu_4 J_E \quad (36)$$

where μ_1, μ_2, μ_3 , and μ_4 represent the weighting coefficients. The optimal path problem is transformed into a minimum optimization problem. It considers four factors, including obstacle avoidance function J_O , path length function J_L , path curvature function J_K , and energy consumption function J_E .

3.4.1. Obstacle avoidance evaluation function

The premise of efficient task execution is safety, so the first consideration is to avoid obstacles in complex environments. Underwater obstacles generally include terrain obstacles and suspended obstacles. In order to ensure safety, an obstacle avoidance loss evaluation function is designed as

$$J_O = 0.5J_{O1} + 0.5J_{O2} \quad (37)$$

where J_{O1} is the loss assessment of terrain obstacle avoidance; J_{O2} denotes an obstacle avoidance loss assessment of suspended obstacles.

$$J_{O1} = \begin{cases} \sum_{i=1}^n \frac{Z_i}{z_i - h}, z_i \in (Z_i + h, +\infty) \\ \text{INF}, z_i \in (-\infty, Z_i + h) \end{cases} \quad (38)$$

$$d_{ij} = \sqrt{(x_i - X_0(j))^2 + (y_i - Y_0(j))^2 + (z_i - Z_0(j))^2} - R(j) \quad (39)$$

$$J_{O2} = \begin{cases} \sum_{i=1}^n \sum_{j=1}^m \frac{1}{d_{ij}}, d_{ij} \in [H, +\infty) \\ \text{INF}, d_{ij} \in (-\infty, H) \end{cases} \quad (40)$$

where Z_i represents the corresponding terrain height under the (x_i, y_i) of the path points (x_i, y_i, z_i) ; h is the safety threshold of terrain obstacle avoidance; d_{ij} signifies the distance from the i th path point to the j th suspended obstacle; H is the safety threshold of suspended obstacles.

3.4.2. Path length evaluation function

In the context of a limited energy supply, minimizing path length becomes imperative to extend the operational duration of AUVs and bolster their task execution capabilities. In such circumstances, a path length evaluation function [25] is designed as

$$J_L = \sum_{i=1}^n \sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2 + (z_i - z_{i-1})^2}. \quad (41)$$

3.4.3. Path curvature evaluation function

A smaller path curvature can reduce the motion loss of AUVs, while a smoother path can enhance the stability and accuracy of control. The sum of the curvature of the path points being small means the path is relatively smoother. The path curvature evaluation function is formulated by accumulating the curvature between adjacent path points

$$\begin{aligned} \mathbf{O} &= (x_i, y_i, z_i) \\ J_K &= \sum_{i=1}^n \frac{\sqrt{|\dot{\mathbf{O}}|^2 |\ddot{\mathbf{O}}|^2 - (\dot{\mathbf{O}} \cdot \ddot{\mathbf{O}})^2}}{\|\dot{\mathbf{O}}\|^3} \end{aligned} \quad (42)$$

3.4.4. Energy evaluation function

Energy is a critical resource for AUV operations, and its depletion can result in mission failure or interruption. The energy loss of AUVs in the ocean environment is primarily influenced by ocean currents and path length. Utilizing ocean currents effectively is paramount in reducing energy loss, so it is necessary to design an energy loss evaluation function

$$J_E = -\rho \sum_{i=1}^n \frac{\alpha \beta}{|\alpha| |\beta|} + \chi J_L \quad (43)$$

where $\alpha = (\dot{x}_i, \dot{y}_i, \dot{z}_i)$ represents the current tangent direction, while $\beta = (V_{xi}, V_{yi}, V_{zi})$ denotes the flow field state corresponding to the current path point (x_i, y_i, z_i) ; ρ and χ represent the energy loss weights of flow field and path length.

3.5. Constraint condition and problem model

The 3D path planning works only in the feasible region, so boundary constraints are set as

$$\begin{cases} x_{\min} \leq x \leq x_{\max} \\ y_{\min} \leq y \leq y_{\max} \\ z_{\min} \leq z \leq z_{\max} \end{cases} \quad (44)$$

To sum up, the 3D path planning problem of AUVs is described as a minimum optimization problem, expressed as

$$\begin{aligned} \min \{ & J(F) = \mu_1 J_O + \mu_2 J_L + \mu_3 J_K + \mu_4 J_E \} \\ \text{s.t. } \begin{cases} & x_{\min} \leq x \leq x_{\max} \\ & y_{\min} \leq y \leq y_{\max} \\ & z_{\min} \leq z \leq z_{\max} \end{cases} \end{aligned} \quad (45)$$

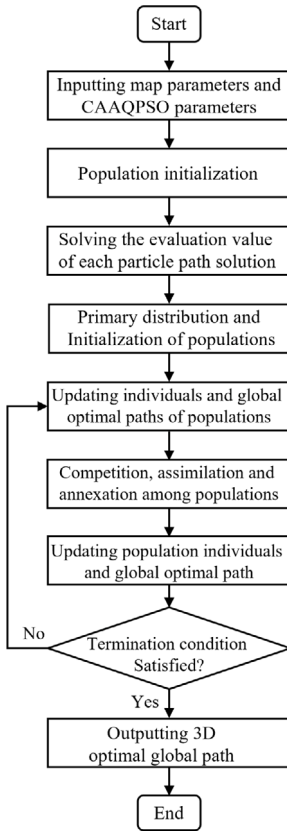


Fig. 4. Schematic diagram of solving path planning.

3.6. The 3D path planning process

The specific process for solving the 3D path planning problem based on the proposed method is as follows:

- (1) Input map parameters, starting point, and endpoint coordinates.
- (2) Take the intermediate path point as the value to be solved.
- (3) Take the evaluation function as the path evaluation index.
- (4) Use CAAQPSO algorithm to update the path points. The specific process is shown in Fig. 4.
- (5) Obtain the global optimal path.

4. Simulation

4.1. Simulation configuration

Seven algorithms, namely PSO [35], QPSO [30], ICFPSO [25], SDPSO [29], HCP SOA [28], Dandelion Optimizer (DO) [38], and Artificial hummingbird algorithm (AHA) [39], are compared with proposed methods under different flow fields in a complex environment. The algorithm parameters and map configuration are shown in Table 1. The schematic diagram of the adaptive law is shown in Fig. 5.

4.2. Simulation results and analysis

Because the initial data of each population allocation is saved, the actual competitive-assimilation process data shows a lag. Fig. 6 shows the typical iteration curve of CAAQPSO and CAA*QPSO, and it can be clearly seen that there is a different degree of decrease every 40 iterations, in which the large decrease highlights the advantage of CAAQPSO algorithm in jumping out of local optimization to find a better solution. And CAA*QPSO is almost the same in updating strategy.

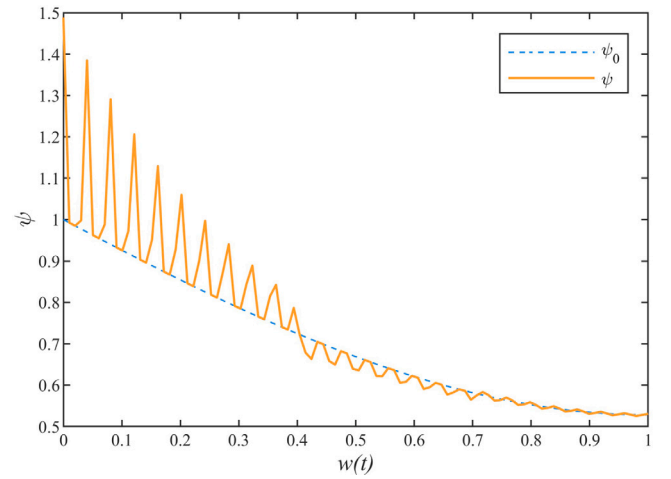


Fig. 5. The schematic diagram of the adaptive CE coefficient ($w(t)$ represents the current iteration period; the blue dotted line is the fluctuation center of the adaptive CE coefficient ψ_0 , and the orange solid line is the actual value of the actual CE coefficient.).

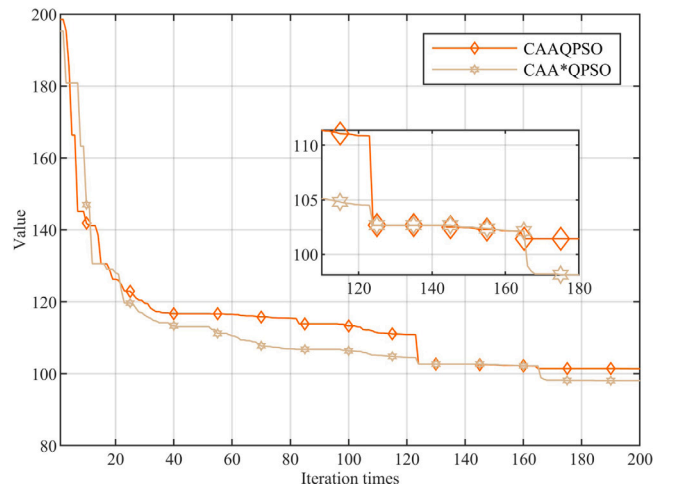


Fig. 6. Iterative features of CAAQPSO and CAA*QPSO.

The general flow field in the environment is more likely to change. Therefore, $V_{x0} = -0.5$ m/s and $V_{y0} = 0.15$ m/s are designed as the first kind environment Env_1 and the second kind environment Env_2 , respectively. And the flow field presents a countercurrent state and a downstream state. Figs. 7 and 8 show the path planning results and the iterative change curves of evaluation values in the first kind environment. Furthermore, the path planning results and evaluation values in the second kind environment are compared in Figs. 9 and 10. Fig. 11 and Table 2 show the results of 100 tests in two environments.

From the path shown in Fig. 7, it can be seen that HCP SOA, CAAQPSO, and CAA*QPSO can all get shorter and smoother paths, but the turning angle of HCP SOA is relatively larger than the other two at the position of $x = 50$ m. The curve in Fig. 8 shows that PSO falls into local optimization prematurely, CAAQPSO can find the relative optimal solution first, and CAA*QPSO finds a better solution than all other algorithms.

Fig. 9 describes that CAAQPSO and CAA*QPSO can still get paths that are more in line with the navigation law of UAVs in different environments, and HCP SOA can find similar paths. However, Fig. 10 shows that CAAQPSO can find a small evaluation value, and CAA*QPSO can further improve the local optimization ability based on it.

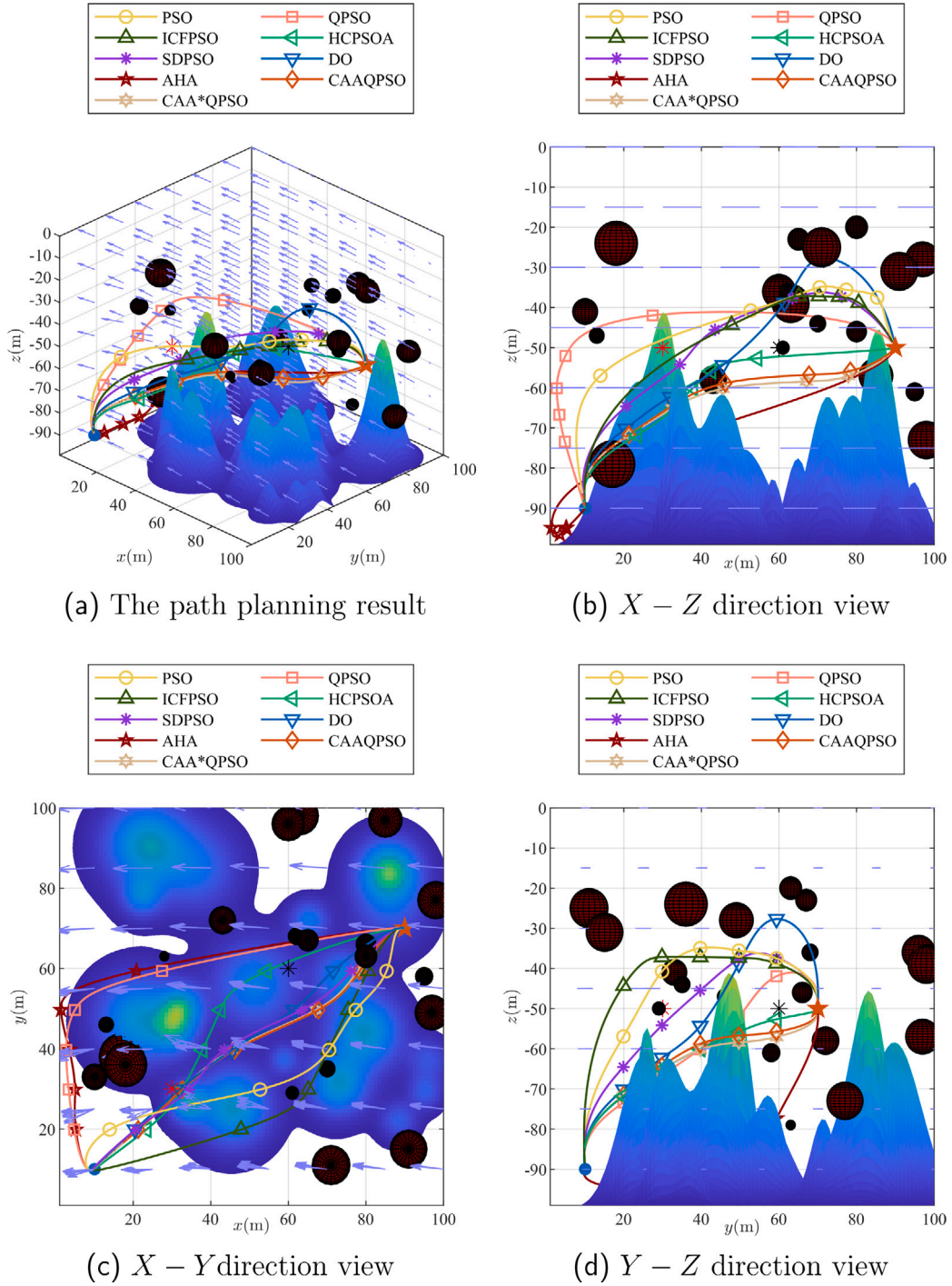


Fig. 7. The paths of the first kind of environment.

The results (Figs. 8 and 10) of heuristic algorithms DO and AHA show that there are limitations in solving path planning problems, as they all fall into local optimization.

In the two box diagrams shown in Fig. 11, it can be clearly seen that CAAQPSO has the least discrete value, and its overall evaluation value is smaller than PSO, QPSO, ICFPSO, SDPSO, DO, and AHA. It is shown that HCFPSOA has a stronger local optimization ability in both environments, but it has large discrete values. In addition, CAA*QPSO, which also uses the COA, has faster convergence and better stability than HCFPSOA. The data in Table 2 can support the above statement. Although the optimal value of HCFPSOA is slightly better than the other algorithms, because of the slow convergence speed of the

COA, the simpler CAAQPSO algorithm is more suitable for applications. Without considering the running speed of the algorithm, CAA*QPSO has outstanding advantages for all algorithms.

Table 2 demonstrates the superiority of the proposed strategy through evaluations and time performance in two environments. Compared to the original QPSO, CAAQPSO with the proposed strategy increases single-run time by only 0.45449 s (3.84%) and 0.67314 s (5.37%), respectively, while achieving better performance (1.51% and 3.44% mean reduction) and higher stability (55.41% and 86.88% standard deviation reduction). This confirms limited computational overhead increase relative to performance gains. Wilcoxon rank-sum and Friedman tests show that at significance level of $\alpha = 0.05$,

Table 1
Algorithms parameters and map configurations.

Algorithm	Parameter
Common parameters	Map range = (100 m, 100 m, -100 m); $M = 100$; $D = 5$; $T = 200$; $\rho_1 = \rho_2 = 0.35$; $\Psi_1 = 0.4$; $\Psi_2 = 0.7$; $\Xi_1 = \Xi_2 = 0.4$; $\Xi_3 = 0.5$; $\zeta_1 = 0.4$; $\zeta_2 = -0.4$; $\zeta_3 = -2.5$; $t_1 = t_2 = t_3 = 4$; $\kappa_1 = \kappa_2 = \kappa_3 = 50$; $(x_0, y_0, z_0) = (10 \text{ m}, 10 \text{ m}, -90 \text{ m})$; $(x_N, y_N, z_N) = (90 \text{ m}, 70 \text{ m}, -50 \text{ m})$; $(\mu_1, \mu_2, \mu_3, \mu_4) = (0.21, 0.39, 0.19, 0.21)$.
PSO	$\omega = 1.2$; $c_1 = c_2 = 2.0$.
QPSO	$\psi = [0.5, 1]$.
ICFPSO	$\omega = [0.4, 0.9]$; $\sigma = 0.3$; $c_1 = c_2 = 2.05$.
SDPSO	$\omega = 1.2$; $c_1 = c_2 = 2.0$.
HCP SOA	$\omega = [0.4, 0.9]$; $c = [0.5, 0.9]$; $c_1 = c_2 = 1.5$; $N_p = 20$; $N_e = 5$.
DO	$\alpha \in [0, 1]$; $k \in [0, 1]$
AHA	Migration coefficient = M
CAAQPSO	$k = 5$.
CAA*QPSO	$k = 5$; $N_e = 5$.

Table 2
Comparison of path value.

Algorithm	Env ₁			Env ₂		
	J_{Opt}	J_M	J_{SD}	J_{Opt}	J_M	J_{SD}
PSO [35]	118.7459 1345.413 s, [6.87]	123.9943 ^{†,‡}	3.6695	99.7662 1278.965 s, [6.15]	105.8145 ^{†,‡}	3.7283
QPSO [30]	115.7067 1208.852 s, [4.57]	119.9881 ^{†,‡}	2.9385	98.0969 1254.912 s, [4.86]	103.8105 ^{†,‡}	3.9131
ICFPSO [25]	117.6274 1314.435 s, [5.94]	122.2570 ^{†,‡}	3.5168	99.4961 1293.832 s, [5.54]	105.0242 ^{†,‡}	4.1474
HCP SOA [28]	115.3982 4817.753 s, [1.38]	115.9010 ^{§,≈}	0.6509	97.7475 4593.446 s, [1.20]	98.0077 ^{§,≈}	2.0521
SDPSO [29]	118.0024 1373.552 s, [6.71]	123.9956 ^{†,‡}	4.5536	99.6290 1287.861 s, [6.17]	106.5730 ^{†,‡}	5.3702
DO [38]	115.9617 1445.814 s, [6.3]	124.0190 ^{†,‡}	6.2338	98.1521 1464.758 s, [6.82]	110.0487 ^{†,‡}	7.7399
AHA [39]	122.1573 1228.290 s, [7.82]	125.6661 ^{†,‡}	2.4365	111.0262 1371.679 s, [8.81]	120.5087 ^{†,‡}	3.8128
CAAQPSO	115.7819 1255.301 s, [3.61]	118.1683 [†]	1.3102	97.7968 1322.226 s, [3.53]	100.4364 [†]	1.8006
CAA*QPSO	115.6690 3585.763 s, [1.80]	115.8510 [§]	0.3856	97.7537 3519.272 s, [1.92]	97.8313 [§]	0.0641

Note: The table compares various optimization algorithms and their performances on two different medium size ($M = 150, 200$), and large. The values indicate the results of different indexes including Optimum J_{Opt} , Mean J_M , and Standard Deviation J_{SD} . The best results are highlighted in bold. At a significance level of $\alpha = 0.05$, Wilcoxon rank-sum test [40] results are annotated as superscripts on mean values, where [†] denotes statistically significant superiority of CAAQPSO; [‡] indicates statistical equivalence; [§] represents significant inferiority. For CAA*QPSO, the corresponding markers are ^{†*}, ^{≈*}, and ^{§*}, respectively. The second row of benchmark results presents total computation time over 100 tests alongside Friedman test mean rankings.

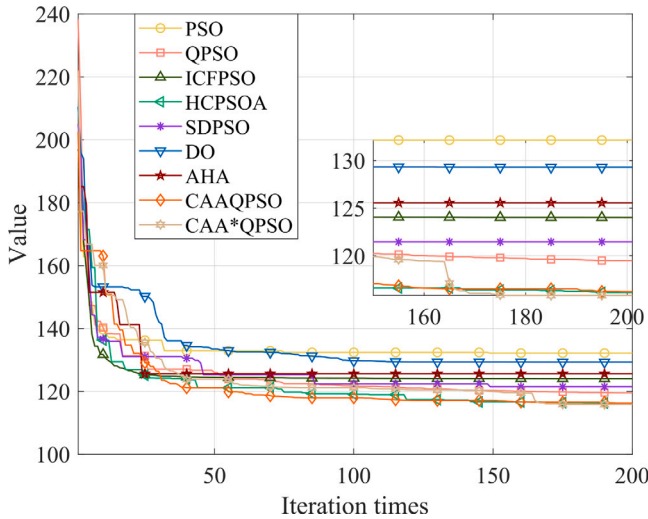


Fig. 8. The value of paths in the first kind of environment.

CAAQPSO has significant advantages over the other algorithms except HCP SOA and CAA*QPSO. Friedman mean ranks further indicate no significant difference between HCP SOA (rank 1.38 and rank 1.20) and CAA*QPSO (rank 1.80 and rank 1.92), yet with 25.57% and 23.38% time reduction, respectively.

In summary, CAAQPSO has obvious advantages in solving the problem that PSO is easy to fall into local optimum and is significantly influenced by particle initialization under the condition of ensuring the easy implementation of the algorithm. Ignoring the running speed, CAA*QPSO not only solves the above problems, but also solves the problem of PSO convergence accuracy. Meanwhile, the stability (Smaller standard deviation) of both algorithms is a key advantage for the optimization problem.

5. Conclusion

To address the issues with swarm optimization algorithms in 3D path planning, which are prone to falling into local optimum and are greatly influenced by particle initialization, a competitive-assimilation strategy was proposed and verified on QPSO in this paper. By combining the proposed strategy with a newly developed adaptive law, the CAAQPSO algorithm was generated, which could solve path planning problems better. To fully improve the local search capability of the

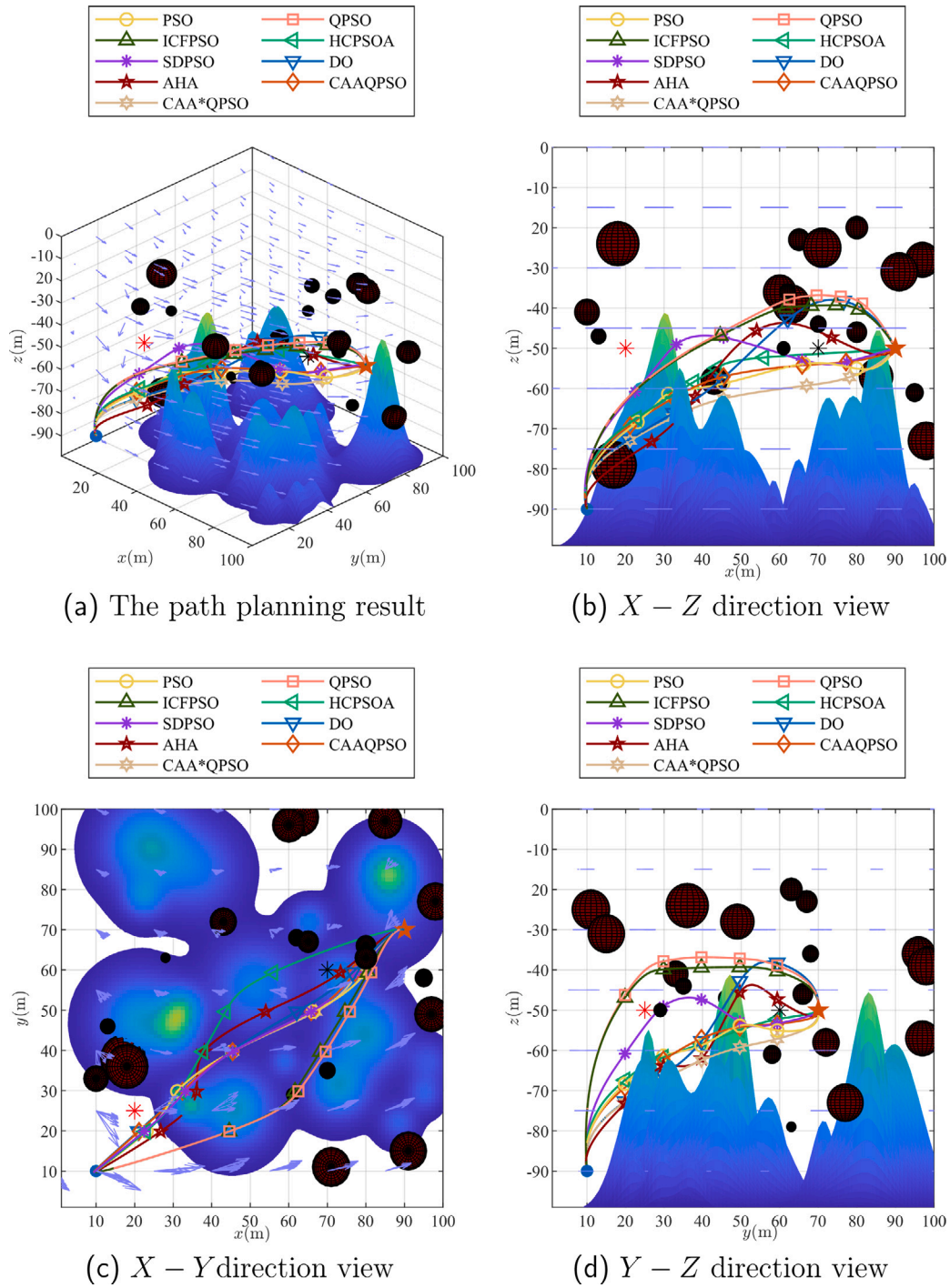


Fig. 9. The paths of the second kind of environment.

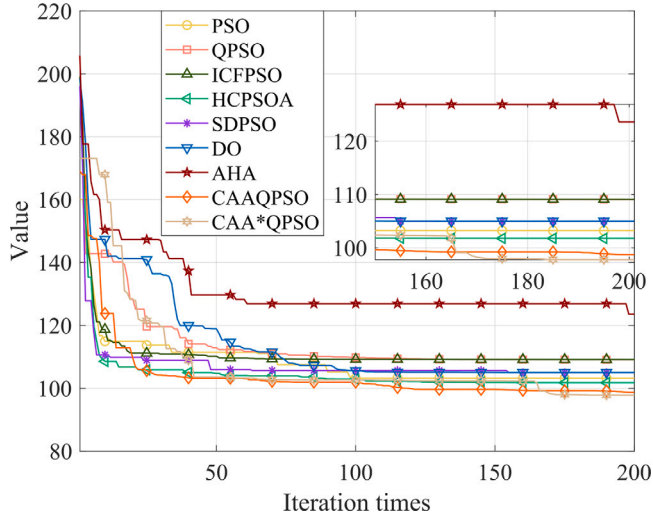
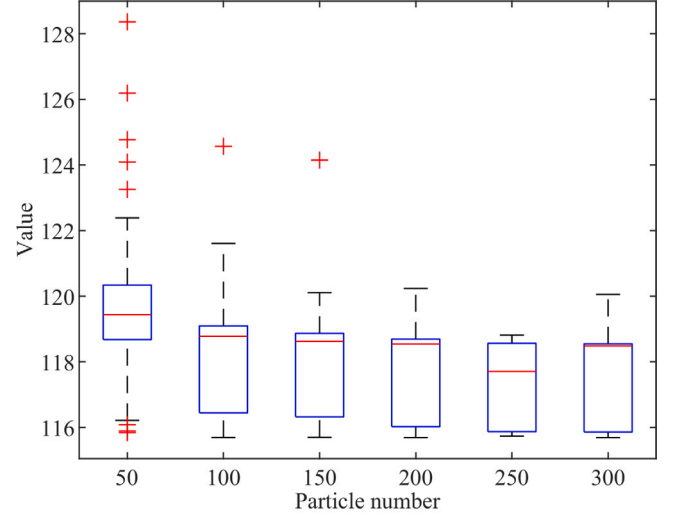
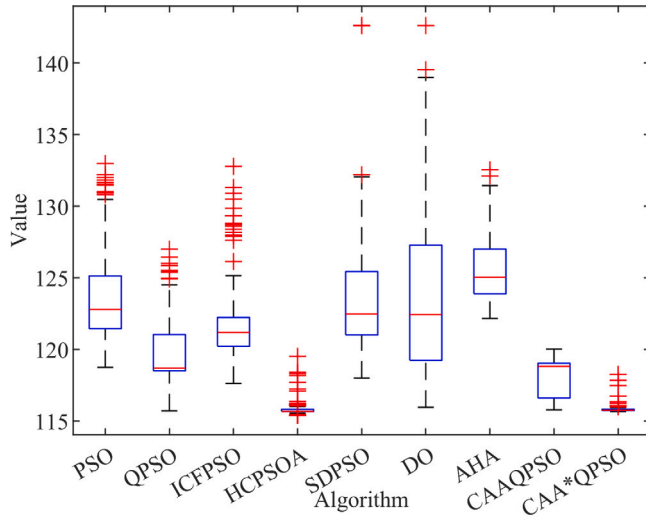


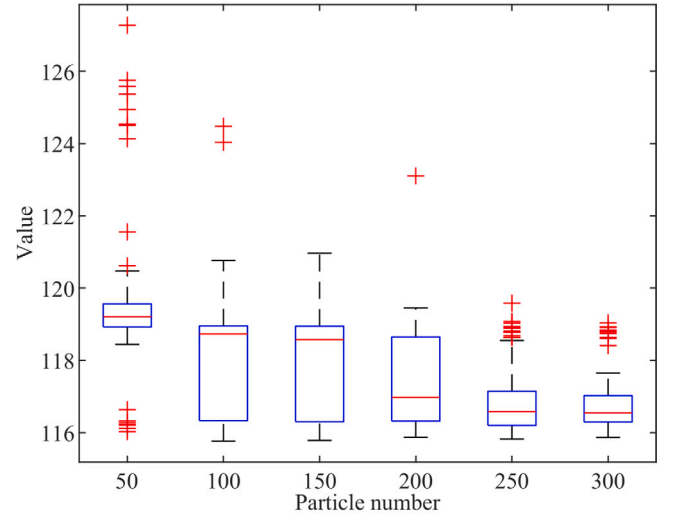
Fig. 10. The value of paths in the second kind of environment.



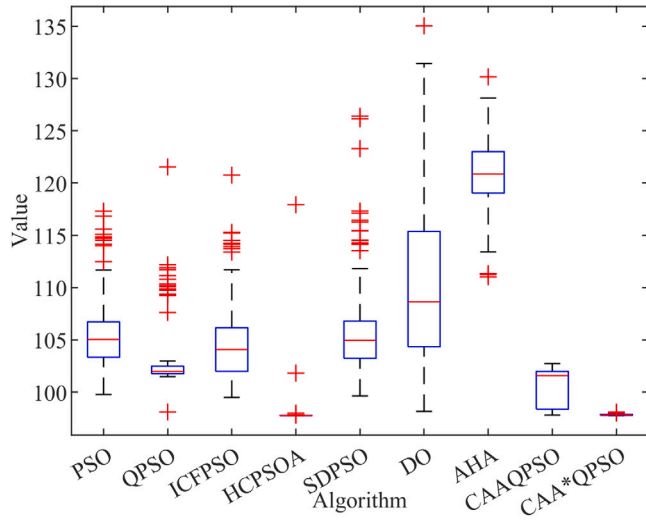
(a) Fixed population allocation quantity



(a) The test in the first kind of environment



(b) Adaptive population allocation quantity



(b) The test in the second kind of environment

Fig. 11. The results of 100 tests.

Fig. 12. Box diagrams of particle number sensitivity test.

algorithm, the CAA*QPSO algorithm was produced by integrating COA. The proposed method demonstrates 1.51% mean improvement in path quality and 55.41% stability gain over state-of-the-art optimizers, with its enhanced variant achieving 3.44% precision boost and 86.88% variance reduction. These statistically validated advancements establish new benchmarks for 3D path planning robustness and solution optimality.

Future research directions should focus on 2 critical aspects. First, comprehensive verification through integration of the planning model with real-world underwater scenarios is essential for practical implementation. Second, the proposed methodology provides significant potential for cross-domain applications, particularly in: (1) intelligent navigation for AUVs, aerial drones, and cross-medium systems, (2) medical procedure optimization of surgical robots, (3) adaptive scheduling in smart manufacturing systems, and (4) management strategies for smart energy management systems.

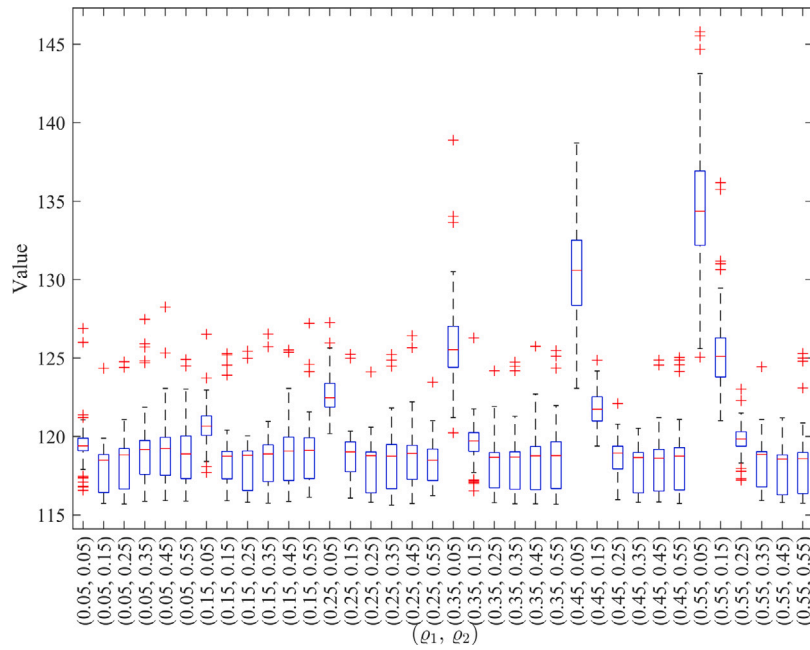


Fig. 13. Box diagram of CE coefficient sensitivity test.

Table 3
Particle number sensitivity test.

Parameter	Total particle number M						
	Index	50	100	150	200	250	300
$k = 5$	J_{Opt}	115.8378	115.6933	115.7004	115.6883	115.7364	115.6881
	J_{M}	119.4366	118.1787	117.9639	117.6716	117.2927	117.5621
	Adaptive m	J_{SD}	1.9994	1.5512	1.4179	1.3534	1.3095
$m = 20$	J_{Opt}	116.0276	115.7647	115.7837	115.8700	115.8206	115.8657
	J_{M}	119.6429	118.0965	117.8335	117.4336	116.9040	116.8101
	Adaptive k	J_{SD}	2.0790	1.6365	1.3559	1.3075	0.9813

Note: The values indicate the results of different indexes including Optimum J_{Opt} , Mean J_M , and Standard Deviation J_{SD} .

CRedit authorship contribution statement

Mengfan Wang: Writing – review & editing, Visualization, Software, Methodology, Conceptualization, Writing – original draft, Validation, Investigation. **Xinglin Chen:** Writing – review & editing, Investigation, Software. **Siyu Liu:** Visualization, Investigation, Software. **Sisindisiwe Nomalanga Ncube:** Writing – review & editing, Investigation. **Ruotong Ming:** Investigation, Writing – review & editing. **Fuqiang Liu:** Writing – review & editing, Supervision, Conceptualization, Validation, Funding acquisition. **Jun Luo:** Funding acquisition, Supervision. **Huayan Pu:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to thank the Chongqing Municipal Science and Technology Bureau (Grant No. CSTB2023TIAD-KPX0057) and the National Natural Science Foundation of China (Grant No. 62033001).

Appendix

The appendix mainly includes the content of parameter sensitivity analysis.

Competitive-assimilation strategy

A competitive-assimilation strategy based on Darwinian evolution theory is proposed to address the limitations of swarm optimization algorithms. This strategy introduces a novel population renewal and distribution scheme, where the key parameters are particle number M and distribution scheme k .

For particle number sensitivity analysis, two methods are implemented in Env_1 : fixed population quantity allocated at $k = 5$ and adaptive population quantity adjusted based on minimum particle number $m = 20$ of population. The experimental analysis is carried out in three scales: small number of particles ($M = 50, 100$), medium size ($M = 150, 200$), and large size ($M = 250, 300$).

Fig. 12 and Table 3 show that with fixed $k = 5$, as the scale increases, the mean and standard deviation slightly decrease, indicating limited performance improvement beyond small-scale scenarios. When k is adaptive, larger scales correspond to increased subpopulations, demonstrating significant performance enhancement with scale expansion.

Adaptive CE coefficient

Sun et al. [36] found that the CE coefficient critically influences QPSO performance. The fluctuation center ψ_0 of the proposed adaptive law plays a crucial role in determining the CE coefficient range. Therefore, parameter sensitivity analysis is conducted for ρ_1 and ρ_2 which determine ψ_0 .

Table 4
CE coefficient sensitivity test.

ρ_1	ρ_2					
	0.05	0.15	0.25	0.35	0.45	0.55
0.05	119.4689 (1.3500)	117.8454 (1.4531)	118.3734 (1.6467)	119.0555 (2.0366)	119.0197 (1.8495)	118.9460 (1.8654)
0.15	120.7702 (1.1723)	118.5002 (1.6724)	118.1808 (1.6773)	118.5525 (1.7449)	118.9574 (1.8933)	118.9476 (1.9077)
0.25	122.7504 (1.2188)	118.7664 (1.4980)	118.0260 (1.4844)	118.4738 (1.9995)	118.6736 (1.7525)	118.3382 (1.4140)
0.35	126.0049 (2.6652)	119.6068 (1.3261)	118.1935 (1.3598)	118.2343 (1.7303)	118.3553 (1.7096)	118.4884 (1.9890)
0.45	130.4341 (2.9011)	121.7766 (1.1318)	118.6818 (1.2060)	117.9803 (1.3022)	118.0684 (1.6844)	118.4609 (2.0205)
0.55	134.5470 (3.9112)	125.3172 (2.5683)	119.8066 (0.8995)	118.2985 (1.3680)	117.8336 (1.3783)	118.0791 (1.9249)

Note: The values indicate the results of different indexes including Mean J_M and Standard Deviation J_{SD} , expressed as $J_M(J_{SD})$.

Values are selected within [0.05, 0.60] for detailed analysis (Fig. 13 and Table 4). Analysis reveals ρ_1 primarily affects the mean value (algorithm convergence), while ρ_2 mainly influences the standard deviation (algorithm stability).

Data availability

The data that has been used is confidential.

References

- [1] A. Sahoo, S.K. Dwivedy, P.S. Robi, Advancements in the field of autonomous underwater vehicle, *Ocean Eng.* 181 (2019) 145–160.
- [2] C. Cheng, Q. Sha, B. He, G. Li, Path planning and obstacle avoidance for AUV: A review, *Ocean Eng.* 235 (2021) 109355.
- [3] D. An, Y. Mu, Y. Wang, B. Li, Y. Wei, Intelligent path planning technologies of underwater vehicles: a review, *J. Intell. Robot. Syst.* 107 (2) (2023) 22.
- [4] K. Gao, M. Gao, M. Zhou, Z. Ma, Artificial intelligence algorithms in unmanned surface vessel task assignment and path planning: A survey, *Swarm Evol. Comput.* 86 (2024) 101505.
- [5] W. Zhang, N. Wang, W. Wu, A hybrid path planning algorithm considering AUV dynamic constraints based on improved A* algorithm and APF algorithm, *Ocean Eng.* 285 (1) (2023) 115333.
- [6] Z. Chu, F. Wang, T. Lei, C. Luo, Path planning based on deep reinforcement learning for autonomous underwater vehicles under ocean current disturbance, *IEEE Trans. Intell. Veh.* 8 (1) (2023) 108–120.
- [7] J. Yang, J. Ni, M. Xi, J. Wen, Y. Li, Intelligent path planning of underwater robot based on reinforcement learning, *IEEE Trans. Autom. Sci. Eng.* 20 (3) (2023) 1983–1996.
- [8] H. Zhang, X. Shi, C. Li, An improved quantum-behaved particle swarm optimization algorithm combined with reinforcement learning for AUV path planning, *J. Robot.* 2023 (2023) 1–11.
- [9] Z. Yan, J. Yan, Y. Wu, S. Cai, H. Wang, A novel reinforcement learning based tuna swarm optimization algorithm for autonomous underwater vehicle path planning, *Math. Comput. Simulation* 209 (2023) 55–86.
- [10] K.S. Gaber, M.K. Singla, Predictive analysis of groundwater resources using random forest regression, *J. Artif. Intell. Metaheuristics* (2025) 11–19.
- [11] M.E. Mohamed, A review on waste management techniques for sustainable energy production, *Metaheuristic Optim. Rev.* (2025) 47–58.
- [12] Z. Zhang, J. Zhang, Y. Zhao, Z. Zhu, J. Guo, A novel hexagonal grid map model and regenerated heuristic factor based strategy for intelligent manufacturing system's AGV path planning problem solving, *Comput. Ind. Eng.* 192 (2024) 110154.
- [13] H. Qin, N. Zhou, S. Han, Y. Xue, An environment information-driven online bi-level path planning algorithm for underwater search and rescue AUV, *Ocean Eng.* 296 (2024) 116949.
- [14] R. Meng, X. Cheng, Z. Wu, X. Du, Improved ant colony optimization for safe path planning of AUV, *Heliyon* 10 (7) (2024) e27753.
- [15] K. Hao, J. Zhao, Z. Li, Y. Liu, L. Zhao, Dynamic path planning of a three-dimensional underwater AUV based on an adaptive genetic algorithm, *Ocean Eng.* 263 (2022) 112421.

- [16] B. Li, J. Mao, S. Yin, L. Fu, Y. Wang, Path planning of multi-objective underwater robot based on improved sparrow search algorithm in complex marine environment, *J. Mar. Sci. Eng.* 10 (11) (2022) 1695.
- [17] Y. Huang, Y. Li, Z. Zhang, Q. Sun, A novel path planning approach for AUV based on improved whale optimization algorithm using segment learning and adaptive operator selection, *Ocean Eng.* 280 (2023) 114591.
- [18] F. Sui, X. Tang, Z. Dong, X. Gan, P. Luo, J. Sun, ACO+PSO+A*: A bi-layer hybrid algorithm for multi-task path planning of an AUV, *Comput. Ind. Eng.* 175 (2023) 108905.
- [19] E.-S.M. El-kenawy, N. Khodadadi, S. Mirjalili, A.A. Abdelhamid, M.M. Eid, A. Ibrahim, Greylag Goose optimization: Nature-inspired optimization algorithm, *Expert Syst. Appl.* 238 (2024) 122147.
- [20] S.P. Sahoo, B. Das, B.B. Pati, F.P. Garcia Marquez, I. Segovia Ramirez, Hybrid path planning using a bionic-inspired optimization algorithm for autonomous underwater vehicles, *J. Mar. Sci. Eng.* 11 (4) (2023) 761.
- [21] R. Alkanhel, E.-S.M. El-kenawy, A.A. Abdelhamid, A. Ibrahim, M.A. Alohali, M. Abotaleb, D.S. Khafaga, Network intrusion detection based on feature selection and hybrid metaheuristic optimization, *Comput. Mater. Contin.* 74 (2) (2023) 2677–2693.
- [22] J. Wen, J. Yang, T. Wang, Path planning for autonomous underwater vehicles under the influence of ocean currents based on a fusion heuristic algorithm, *IEEE Trans. Veh. Technol.* 70 (9) (2021) 8529–8544.
- [23] H. Myriam, A. A. Abdelhamid, E.-S.M. El-kenawy, A. Ibrahim, M.M. Eid, M.M. Jamjoom, D.S. Khafaga, Advanced meta-heuristic algorithm based on particle swarm and al-biruni earth radius optimization methods for oral cancer detection, *IEEE Access* 11 (2023) 23681–23700.
- [24] X. Li, S. Yu, Comparison of biological swarm intelligence algorithms for AUVs for three-dimensional path planning in ocean currents' conditions, *J. Mar. Sci. Technol.* 28 (4) (2023) 832–843.
- [25] X. Li, S. Yu, Three-dimensional path planning for AUVs in ocean currents environment based on an improved compression factor particle swarm optimization algorithm, *Ocean Eng.* 280 (2023) 114610.
- [26] C. Lu, J. Yang, B.J. Leira, Q. Chen, S. Wang, Three-dimensional path planning of deep-sea mining vehicle based on improved particle swarm optimization, *J. Mar. Sci. Eng.* 11 (9) (2023).
- [27] C. Huang, X. Zhou, X. Ran, J. Wang, H. Chen, W. Deng, Adaptive cylinder vector particle swarm optimization with differential evolution for UAV path planning, *Eng. Appl. Artif. Intell.* 121 (2023) 105942.
- [28] H. Gupta, O.P. Verma, A novel hybrid coyote-particle swarm optimization algorithm for three-dimensional constrained trajectory planning of unmanned aerial vehicle, *Appl. Soft Comput.* 147 (2023) 110776.
- [29] Z. Yu, Z. Si, X. Li, D. Wang, H. Song, A novel hybrid particle swarm optimization algorithm for path planning of UAVs, *IEEE Internet Things J.* 9 (22) (2022) 22547–22558.
- [30] J. Sun, W. Xu, B. Feng, A global search strategy of quantum-behaved particle swarm optimization, in: *IEEE Conference on Cybernetics and Intelligent Systems*, 2004, vol. 1, 2004, pp. 111–116.
- [31] X. Li, W. Fang, S. Zhu, X. Zhang, An adaptive binary quantum-behaved particle swarm optimization algorithm for the multidimensional knapsack problem, *Swarm Evol. Comput.* 86 (2024) 101494.
- [32] X. Lu, G. He, Enhanced QPSO driven by swarm cooperative evolution and its applications in portfolio optimization, *Swarm Evol. Comput.* 94 (2025) 101872.
- [33] Z. Qi, Z. Shao, Y.S. Ping, L.M. Hiot, Y.K. Leong, An improved heuristic algorithm for UAV path planning in 3D environment, in: *2010 Second International Conference on Intelligent Human-Machine Systems and Cybernetics*, vol. 2, 2010, pp. 258–261.
- [34] B. Garau, A. Alvarez, G. Oliver, AUV navigation through turbulent ocean environments supported by onboard H-ADCP, in: *Proceedings 2006 IEEE International Conference on Robotics and Automation*, 2006, ICRA 2006, 2006, pp. 3556–3561.
- [35] J. Kennedy, R. Eberhart, Particle swarm optimization, in: *Proceedings of ICNN'95 - International Conference on Neural Networks*, vol. 4, 1995, pp. 1942–1948.
- [36] J. Sun, W. Fang, X. Wu, V. Palade, W. Xu, Quantum-behaved particle swarm optimization: Analysis of individual particle behavior and parameter selection, *Evol. Comput.* 20 (3) (2012) 349–393.
- [37] J. Pierzan, L. Dos Santos Coelho, Coyote optimization algorithm: A new metaheuristic for global optimization problems, in: *2018 IEEE Congress on Evolutionary Computation, CEC*, 2018, pp. 1–8.
- [38] S. Zhao, T. Zhang, S. Ma, M. Chen, Dandelion optimizer: A nature-inspired metaheuristic algorithm for engineering applications, *Eng. Appl. Artif. Intell.* 114 (2022) 105075.
- [39] W. Zhao, L. Wang, S. Mirjalili, Artificial hummingbird algorithm: A new bio-inspired optimizer with its engineering applications, *Comput. Methods Appl. Mech. Engrg.* 388 (2022) 114194.
- [40] W. Xu, C. Chen, S. Ding, P.M. Pardalos, A bi-objective dynamic collaborative task assignment under uncertainty using modified MOEA/D with heuristic initialization, *Expert Syst. Appl.* 140 (2020) 112844.